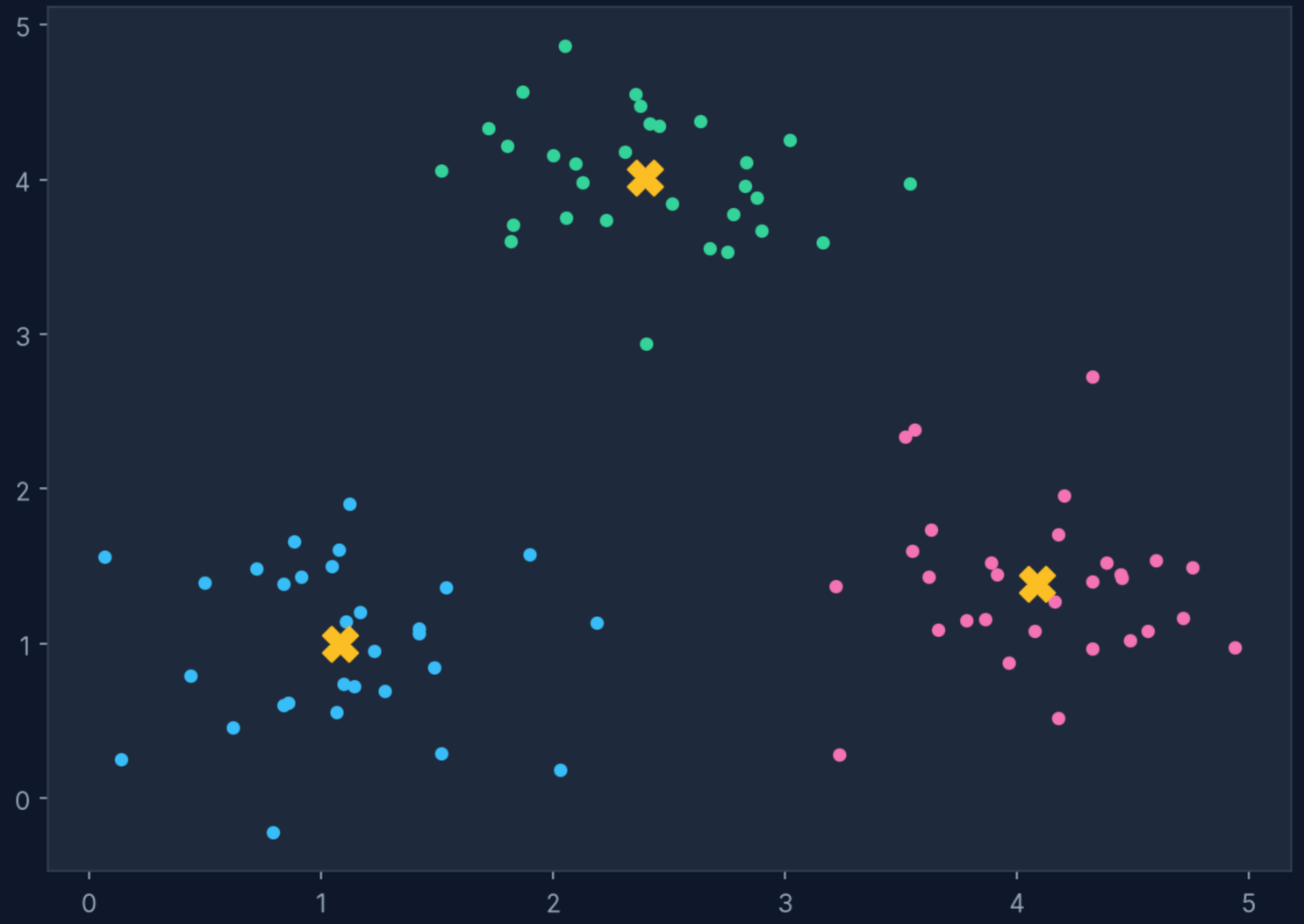
k-Means

Clustering with k-means++

The problem

- Unlabeled data
- Group similar points together
- k is chosen in advance

Intuition



Each cluster has a centre

Points join the nearest centre

Centres move to their members'

Objective and Lloyd's algorithm

$$\min \sum_{i} \|x_i - \mu_{c(i)}\|^2$$

- Assign: each point → nearest centre
- Update: centre ← mean of its points
- Repeat until centres stop moving

k-means++ initialisation

- First centre: random point
- Next: sample with probability $\propto D(x)^2$
- Spreads centres out → better, faster results

Implementation

```
def _init(self, X, rng):
    centers = [X[rng.integers(len(X))]]
    for _ in range(1, self.k):
        d2 = np.min(((X[:, None] - np.array(centers)[None]) ** 2).sum(-1), axis=1)
        centers.append(X[rng.choice(len(X), p=d2 / d2.sum())])
    return np.array(centers)

def fit(self, X):
    X = np.asarray(X, float)
    rng = np.random.default_rng(self.seed)
    self.centers = self._init(X, rng)
    for _ in range(self.max_iter):
        labels = self.predict(X)
        new = np.array([X[labels == j].mean(0) if np.any(labels == j) else self.centers[j] for j in range(self.k)])
        shift = np.abs(new - self.centers).max()
        self.centers = new
        if shift < self.tol:
            break
    self.labels_ = self.predict(X)
    self.inertia_ = float(((X - self.centers[self.labels_]) ** 2).sum())
    return self
```

Complexity & trade-offs

- $O(n \cdot k \cdot d)$ per iteration
- Local minima: run several seeds
- Assumes round, similar-size clusters
- Pick k with the elbow method or silhouette

Recap

- Alternate assign and update
- Minimises within-cluster squared distance
- k-means++ for a good start
- Fast and simple